Université Moulay Ismail de Meknès
Faculté des Sciences
Département d'Informatique
PE ISOC / S5

Module : Compilation

Prof: Abdelaaziz Hessane

Année Universitaire: 2025/2026

Chapitre 2: Analyse lexicale

2.1. Définition

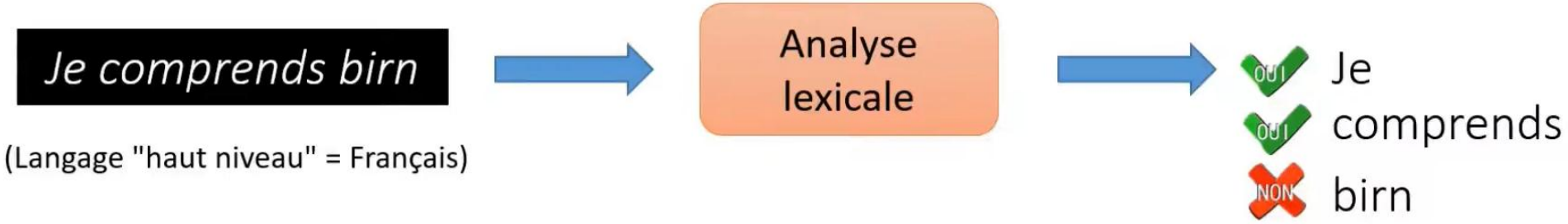
2.2 Expressions régulières (Partie 1)

- Définition
- Alphabet, mot et langage
- Propriétés des ER
- Syntaxe des ER

2.3 Automates à états finis (Partie 2)

2.1. Définition

L'analyse lexicale s'assure que tous les **mots**, utilisés dans le code source, appartiennent au langage haut niveau.



2.1. Définition : Notion de lexème

Lexème :

La première étape de l'analyse lexicale consiste à **détecter les mots composants du code source.**

- Pour chaque langage de haut niveau, une description physique est utilisée pour détecter les mots de code source.

Par exemple, pour le français :

- Un mot est un ensemble de symboles de l'alphabets (du français) délimité par :
  - Deux espaces;
  - Un espace et un symbole de ponctuation
  - Etc.
- D'une façon plus générale, en compilation on utilise **Lexème** au lieu de mot.

Lexème = Unité lexicale

2.1. Définition : Notion de lexème

Un **lexème** est donc : **une séquence finie de symboles de l'alphabet du langage.**

Exemple: Quels sont les lexèmes du code C# suivant:

```
while (ip<z)
    ++ip;
```

While	z
(	)
ip	++
<	ip
	;

N.B: Les espaces, retours à la ligne sont ignorés.

2.1. Définition : Notion de Jeton (Token)

- La deuxième étape de l'analyse lexicale consiste à valider si chaque lexème appartient ou non au langage.
- Il est impossible d'énumérer tous les lexèmes possibles d'un langage (nom des variables qu'on peut utiliser!)

↳ La validation s'effectue en définissant toutes les **parties logiques du ficher source (Jeton)**, puis associer chaque jeton à un lexème.

Le lexème qui ne peut pas être associé à aucun jeton n'appartient pas donc au langage.

2.1. Définition : Notion de Jeton (Token)

- Un jeton représente une partie logique du fichier source:
  - Un mot clé : if, while ...
  - Une variable : ip, z ...
  - Un opérateur : +, < ...
  - Un littéral : true, "Maroc" ...
  - ...
- Chaque jeton peut avoir des attributs optionnels qui sont des informations supplémentaires tirées du texte:
  - Par exemple une valeur numérique : les trois caractères "137" sont un lexème associé au jeton entier qui a une valeur numérique égale à 137

2.1. Définition : Notion de Jeton (Token)

- Un jeton est associé à un ensemble de lexèmes (fini ou infini)
  - Le jeton du mot clé "wile" à l’ensemble {while}
  - Le jeton "entier" à l’ensemble {0,1,2,...}

- Comment associer chaque type de jeton à un ensemble de lexèmes potentiellement infini (Comment décrire le langage) ?
  - **Expressions régulières**
  - **Automate**
  - **Grammaire**

2.2. Expressions régulières : Définition

- Une expression régulière (ER) est une chaîne de caractères qui **décrit** un ensemble (fini ou infini) de chaînes de caractères
- Une ER permet donc de **vérifier** si une chaine de caractères appartient, ou non, à l'ensemble de chaines de caractères décrit.

Expression régulière ≃ Un moule

- Décrit un ensemble de chocolats
- Vérifie si un chocolat appartient à cet ensemble

2.2. Expressions régulières : Définition

- Une ER permet d'énumérer (quand c'est possible!) de manière exhaustive toutes les chaines d'un format donné
- Exemple : Soit les chaines de caractères : abcd; aaa; a; ad; ...
- Format : ?

Toutes les chaines de caractères qui commencent par "a "

- Expression régulière :

a.*

Une ER permet de **spécifier formellement un modèle**

2.2. Expressions régulières : Définition

- L'écriture d'une ER suit une **syntaxe bien précise**

Par exemple : (a*, aa|(b|c)...)

- Les ER permettent donc:
  - ✓**Vérifier** si une chaine de caractères respecte la syntaxe de l'expression
  - ✓**Retrouver** (**Identifier**) des sous-chaines qui respectent une certaine syntaxe à l'intérieur d'une chaine de caractères
  - ✓**Remplacer** des sous-chaines qui respectent une certaine syntaxe à l'intérieur d'une chaine de caractères.

2.2. Expressions régulières : Définition

- Pour résumer :
  - Les ER sont **un outil qui permet de décrire les mots d'un langage**
  - Les langages décrits par les ER sont appelés **langages réguliers**

2.2. Expressions régulières : Alphabet

- Un alphabet, souvent référencé par ∑, est un ensemble de symboles

Exemples : {0,1}, {0,1,..,9}, {a,b,..,z}, ...

- Généralement, les caractères vides ne font pas partie des alphabetss: espace, tabulation, retour chariot

2.2. Expressions régulières : Mot

- Un **mot** sur un alphabet ∑ est une **séquence finie de symboles de ∑**

Exemple : 01, 00111, 011101011101, 11 sont des mots de l'alphabet {0,1}

La chaine vide est représentée par ε

2.2. Expressions régulières : Opérations sur les mots

- **Concaténation:** la concaténation de deux mots x et y donne lieu a un mot xy

Exemple : **00111** concaténé avec **011011101** donne :

00111 011011101

- **Puissance:**
  - $\mathbf{X^0 = \varepsilon}$, $\mathbf{X^n = XX^{n-1}}$ Si n>0 (Exemple : 00111^3 = 00111001110011 1)

2.2. Expressions régulières : Opérations sur les mots

Exercice :

Soit un alphabet ∑={a,b}, trouver:

1. L'ensemble de tous les mots (chaines) construits à partir de l'alphabet ∑ de longueur 2.
2. L'ensemble de tous les mots (chaines) construits à partir de l'alphabet ∑ de longueur 3.
3. L'ensemble de tous les mots (chaines) construits à partir de l'alphabet ∑ qui commencent par a

2.2. Expressions régulières : Opérations sur les mots

Solution :

Soit un alphabet ∑={a,b}, trouver:

1. L'ensemble de tous les mots (chaines) construits à partir de l'alphabet ∑ de longueur 2.

L = {aa, ab, ba, bb} = ∑ ∑ = $\sum^2$

2. L'ensemble de tous les mots (chaines) construits à partir de l'alphabet ∑ de longueur 3.

$\sum^3$ = $\sum^2$ ∑ = {a,b}.{aa, ab, ba, bb} → L = {aaa, aab, aba, abb, baa, bab, bba, bbb}

3. L'ensemble de tous les mots (chaines) construits à partir de l'alphabet ∑ qui commencent par a

L = {a, aa, ab, aaa, aab, aba, abb, aaaa…}

2.2. Expressions régulières : Langage

- Un **langage** sur un alphabet ∑ est **l'ensemble de tous les mots (chaines) construits à partir de l'alphabet ∑**

Quelques cas triviaux:

✓Langage vide **Ø**

✓Langage réduit au mot vide **{ε}**

2.2. Expressions régulières : Opérations sur les langages

- **Union:**

Dénomination	Notation	Définition
Union de L et M	L **U** M	{x | x de L **ou** x de M}

Exemple : Soit $\sum = \{a,b,c\}$, $L_1 = \{a,b\}$, et $L_2 = \{bac, b, c\}$

$L_1 U L_2 = \{a, b, bac, c\}$

2.2. Expressions régulières : Opérations sur les langages

- **Concaténation:**

Dénomination	Notation	Définition
Concaténation de L et M	LM	{xy | x de L et y de M}

Exemple : Soit $\sum = \{a,b,c\}$, $L_1 = \{a,b\}$, et $L_2 = \{bac, b, a\}$

$L_1L_2 = \{abac, ab, aa, bbac, bb, ba\}$

2.2. Expressions régulières : Opérations sur les langages

- **Fermeture de Kleene:**

Dénomination	Notation	Définition
Fermeture de **Kleene** de **L**	L^*	$\{x_1x_2...x_n \mid x_i$ de L, i de N et i>=0$\}$

- **Fermeture Positive**

Dénomination	Notation	Définition
Fermeture positive de **L**	L^+	$\{x_1x_2...x_n \mid x_i$ de L, i de N et i>=1$\}$

Exemple : Soit $\sum = \{a\}$, $L = \{a\}$

$L^0 = \{\varepsilon\}$, $L^1 = \{a\}$, $L^2 = \{aa\}$, $L^3 = \{aaa\}$,...

$L^+ = \{a,aa,aaa,aaaa,...\}$, et $L^* = \{\varepsilon,a,aa,aaa,aaaa,...\}$,

2.2. Expressions régulières : Propriétés

Soit un alphabet ∑, une ER définie sur ∑ ainsi que les langages qu'elle définit sont comme suit:

1. Ø, ε sont des ER qui respectivement définissent les langages Ø, {ε}
2. Pour tous "a" dans ∑, "a" est une expression régulière qui définit le langage {a}

2.2. Expressions régulières : Propriétés

3. Si x et y sont des ER qui définissent respectivement les langages L(x) et L(y), alors:

a) `(x)|(y)` est une expression régulière définissant le langage $L(x) \cup L(y)$

b) `(x)(y)` est une expression régulière définissant le langage $L(x)L(y)$

c) `(x)*` est une expression régulière définissant le langage $(L(x))^*$

d) `(x)` est une expression régulière définissant le langage $L(x)$

2.3. Expressions régulières : Syntaxe

- Une ER est composée d'une ou plusieurs **motifs**
- Chaque motif sert à vérifier une partie de la chaine de caractère

<u>Dans une ER, on trouve généralement:</u>

- Des littéraux
- Des métacaractères
- Des classes de caractères
- Les ancres
- Les quantificateurs
- les groupes et captures

2.3. Expressions régulières : Syntaxe

- **<u>Littéraux</u>**:

➢ Le composant le plus simple d'une expression régulière est le **littéral**. Un littéral correspond simplement à lui-même dans le texte.

➢ Par exemple, l'expression régulière **abc** correspondra exactement à la chaîne "abc" dans le texte recherché.

➢ Les littéraux sont utiles pour des recherches simples où vous savez précisément ce que vous cherchez.

2.3. Expressions régulières : Syntaxe

- **<u>Métacaractères</u>**:

Les métacaractères sont des caractères spéciaux qui ont une signification particulière dans une expression régulière.

Contrairement aux littéraux, ils ne sont pas interprétés comme eux-mêmes, mais comme des commandes qui modifient le comportement de l'expression.

2.3. Expressions régulières : Syntaxe

- **<u>Métacaractères</u>**: Voici quelques métacaractères courants :

Métacaractère	Rôle
[]	Spécifier un ensemble (classe) de caractères parmi lesquels un seul sera sélectionné
\	Ignorer le caractère spécial qui suit (caractère d'échappement). Le considérer comme n'importe quel autre littéral
^	Reconnaître tout caractère sauf ce qui suit (négation) ou indiquer le début de la ligne
$	Fin de ligne
.	Remplace n'importe quel caractère sauf la fin de ligne
-	Intervalle
?	L'élément qui suit peut se répéter 0 ou 1 fois
+	L'élément qui suit peut se répéter 1 ou n fois
{n,m}	L'élément qui suit doit se répéter au moins n fois et au maximum m fois
{n,}	L'élément qui suit se répète au moins n fois
{n}	Exactement n fois
( )	Définir un groupe (sous expression)
|	Alternative (opérateur OU logique)
*	L'élément qui suit peut se répéter 0 ou n fois

2.3. Expressions régulières : Syntaxe

- **<u>Métacaractères</u>**:

Par exemple, l'expression régulière **a.b** correspondra à …

N'importe quelle chaîne de **trois** caractères où le premier est "a", le dernier est "b" et où le caractère du milieu peut être n'importe quoi sauf un retour à la ligne.

2.3. Expressions régulières : Syntaxe

- **Classes de caractères**:

Les classes de caractères permettent de spécifier un ensemble de caractères possibles à un emplacement donné dans le texte. Une classe de caractères est définie par des **crochets []** et elle correspond à **un seul** caractère parmi ceux listés dans les crochets.

2.3. Expressions régulières : Syntaxe

- **<u>Classes de caractères</u>**:

Exemples:

✓Indiquer qu'un chiffre est compris entre 1 et 5: **[1-5]**
Ce qui est équivalent à écrire: [12345]

✓Indiquer qu'une lettre en minuscule peut être comprise entre a et d: **[a-d]**
Ce qui est équivalent à écrire: [abcd]

✓Indiquer n'importe quelle lettre de l'alphabet : **[a-zA-Z]**

2.3. Expressions régulières : Syntaxe

- **<u>Classes de caractères</u>**:

Remarques:

- Les classes de caractères peuvent aussi être **négatives**, ce qui signifie qu'elles correspondent à n'importe quel caractère qui n'est pas dans la liste. Pour cela, on utilise le caractère **^** au début des crochets.

Exemples :

✓Indiquer qu'un caractère est différent de 1, 2, 3, 4, 5: **[^1-5]**

Ce qui est équivalent à écrire: [^12345]

✓Indiquer qu'un caractère est différent de a, b, c, d: **[^a-d]**

Ce qui est équivalent à écrire: [^abcd]

2.3. Expressions régulières : Syntaxe

- **<u>Classes de caractères</u>**:

Remarques:

1. On utilise le caractère **–** pour la soustraction de classes de caractères.

Exemples :

✓ Indiquer toute lettre sauf l, m, n : **[a-zA-z-[lmn]]**

NB: Ce syntaxe spécifique pour la **soustraction de classes de caractères** (ou "classe de caractères négative") qui n'est **pas standard** et n'est supportée que par quelques moteurs d'ER spécifiques (notamment ceux basés sur **.NET/C#** ou **Java**).

2.3. Expressions régulières : Syntaxe

- **Classes de caractères**:

Remarques:

2. Dans une classe de caractères, à l'exception de \,], ^ et – qui ont une signification particulière, les autres métacaractères n'ont pas à être échappés : Ils sont considérés comme étant des littéraux. Le tableau suivant présente quelques classes de caractères particulières :

Métacaractère	Rôle
\d	[0-9]: un chiffre
\D	[^0-9]: un caractère qui n'est pas un chiffre
\w	[a-zA-Z0-9_]: un caractère alphanumérique ou _
\W	[^\w]: un caractère qui n'est pas alphanumérique
\s	[\r\t\n] : espace, tabulation ou saut de ligne
\S	[^\s] : un caractère qui est différent de l'espace

2.3. Expressions régulières : Syntaxe

- **Ancres**:

Les ancres sont des métacaractères qui servent à définir une position:

- ✓Avant des caractères
- ✓Après des caractères
- ✓Entre des caractères

Exemples :

^a.* : une chaine de caractères qui **commence** par un a

.*a**$** : une chaine qui se **termine** par a

2.3. Expressions régulières : Syntaxe

- **<u>Ancres</u>**:
  - Limites des mots (*word boundaries*), représente la position qui limite un mot (début ou fin) : **\b**
    - ✓ .*ut\b : reconnaîtra salut mais pas salutations
    - ✓ \bs.*c\b : reconnaîtra sac mais pas sacoche
  - Limites des mots (*word boundaries*), contraire de \b, : **\B**

    Représente les positions qui ne correspondent pas à celles de \b

2.3. Expressions régulières : Syntaxe

- **<u>Les quantificateurs</u>**:

Les quantificateurs sont utilisés pour indiquer **combien de fois un élément doit apparaître dans le texte pour correspondre au motif**. Ils sont placés directement après le caractère ou le groupe qu'ils quantifient.

* : Correspond à zéro ou plusieurs occurrences.

+ : Correspond à une ou plusieurs occurrences.

? : Correspond à zéro ou une occurrence.

{n} : Correspond exactement à n occurrences.

{n,} : Correspond à au moins n occurrences.

{n,m} : Correspond à entre n et m occurrences.

2.3. Expressions régulières : Syntaxe

- **<u>Les quantificateurs et groupes</u>**:

Par exemple, l'expression régulière a{2,4} correspondra à :

"aa", "aaa", ou "aaaa".

2.3. Expressions régulières : Syntaxe

- **<u>Les Groupes et Captures</u>**:

Les groupes permettent de capturer des parties spécifiques du texte correspondant à une ER. Les groupes sont définis par des parenthèses **()**. Ces groupes peuvent être utilisés pour extraire des sous-chaînes ou pour appliquer des quantificateurs à des sous-ensembles d'une ER.

Exemples :

- ✓(abc) : Correspond à "abc" et capture cette sous-chaîne.
- ✓(?:abc) : Correspond à "abc" sans capturer la sous-chaîne (groupe non capturant).
- ✓(a|b) : Correspond à "a" ou "b" (alternance).

2.3. Expressions régulières : Syntaxe

- **<u>Les Groupes et Captures</u>**:

Les groupes permettent de capturer des parties spécifiques du texte correspondant à une ER. Les groupes sont définis par des parenthèses **()**. Ces groupes peuvent être utilisés pour extraire des sous-chaînes ou pour appliquer des quantificateurs à des sous-ensembles d'une ER.

Exemples :

- ✓ (abc) : Correspond à "abc" et capture cette sous-chaîne.
- ✓ (?:abc) : Correspond à "abc" sans capturer la sous-chaîne (groupe non capturant).
- ✓ (a|b) : Correspond à "a" ou "b" (alternance).

2.3. Expressions régulières : Syntaxe

- **<u>Les assertions (Lookahead et Lookbehind)</u>**

Les assertions permettent de faire **des correspondances conditionnelles** sans consommer de caractères dans la chaîne de texte. Les deux types d'assertions principales sont lookahead (regard en avant) et lookbehind (regard en arrière).

Exemples :

- ✓ Lookahead positif : **(?=abc)** correspond à une position suivie de "abc", sans inclure "abc" dans le résultat.
- ✓ Lookahead négatif : **(?!abc)** correspond à une position non suivie de "abc".
- ✓ Lookbehind positif : **(?<=abc)** correspond à une position précédée de "abc".
- ✓ Lookbehind négatif : **(?<!abc)** correspond à une position non précédée de "abc".

2.3. Expressions régulières : Syntaxe

Lookahead Positif (?=...)

- **Définition :** "Je veux que ce qui suit soit X, mais je ne veux pas capturer X dans mon résultat."
- **Scénario :** Extraire le nom d'une fonction dans du code, c'est-à-dire un mot immédiatement suivi d'une parenthèse ouvrante (.
- **Regex :** \w+(?=\()
- **Texte :** print("Hello"); variable = 10; main();
- **Résultat (Match) :** print et main (mais pas variable).
- *Note : La parenthèse (n'est pas incluse dans le match.*

2.3. Expressions régulières : Syntaxe

Lookahead Négatif (?!...)

- **Définition :** "Je veux que ce qui suit NE SOIT PAS X."
- **Scénario :** Trouver le mot "Windows" uniquement s'il n'est pas suivi de "95" ou "98" (on cherche les versions modernes).
- **Regex :** Windows(?!(95|98))
- **Texte :** Windows10 est mieux que Windows95.
- **Résultat (Match) :** Le premier Windows (celui de Windows10).
- *Note : Le deuxième Windows est rejeté car il est suivi de "95".*

2.3. Expressions régulières : Syntaxe

Lookbehind Positif (?<=...)

- **Définition :** "Je veux ce qui est précédé par X, sans capturer X."
- **Scénario :** Extraire un prix, c'est-à-dire un nombre précédé du symbole $.
- **Regex :** (?<=\$)\d+
- **Texte :** Le prix est $50 ou 45€.
- **Résultat (Match) :** 50 (uniquement).
- *Note : Le nombre 45 n'est pas pris car il n'a pas de $ devant.*

2.3. Expressions régulières : Syntaxe

Lookbehind Négatif (?<!...)

- **Définition :** "Je veux ce qui N'EST PAS précédé par X."
- **Scénario :** Trouver le mot "Error" (pour une alerte critique) sauf s'il est précédé de "No " (indiquant que tout va bien).
- **Regex :** (?<!No)Error
- **Texte :** Statut : No Error. Alerte : Critical Error détectée.
- **Résultat (Match) :** Le deuxième Error (celui de "Critical Error").
- *Le premier "Error" est ignoré car le moteur voit qu'il est précédé immédiatement par la séquence "No ".*

2.3. Expressions régulières : Syntaxe

- **<u>Les assertions (Lookahead et Lookbehind)</u>**

Exemple :

Par exemple, l'expression **(?<=@)\w+** correspondra à :

Un mot qui suit immédiatement un "@" sans inclure le "@" dans la correspondance.

2.3. Expressions régulières : Exercices

Exercice 1:

Écrire une expression régulière qui valide si une chaîne de caractères est une adresse e-mail.

Exemple d'entrée :

valide : exemple@domaine.com

invalide : exemple@domaine

^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$

2.3. Expressions régulières : Exercices

Exercice 2:

Écrire une ER qui extrait tous les numéros de téléphone au format (xxx) xxx-xxxx ou xxx-xxx-xxxx d'un texte.

Exemple d'entrée :

Texte : "Vous pouvez me contacter au (123) 456-7890 ou 987-654-3210."

\(?\d{3}\)?[-.\s]?\d{3}[-.\s]?\d{4}

2.3. Expressions régulières : Exercices

Exercice 3:

Écrire une ER qui identifie toutes les URLs dans un texte.

Exemple d’entrée :

Texte : “Visitez https://www.exemple.com ↗ ou http://exemple.org ↗ pour plus d’informations.”

https?:\/\/[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}([\/\w.-]*)*\/?